

Міністерство освіти і науки України
Карпатський національний університет ім. Василя Стефаника

кафедра КНІС

ЛАБОРАТОРНА РОБОТА № 4
з дисципліни “Теорія програмних систем”
“ПОБУДОВА ДІАГРАМ КЛАСІВ”

виконала:
студентка групи ІСТ-31
Демяник Т. А.

перевірила:
Штаєр Л.О.

Івано-Франківськ

2026

ЛАБОРАТОРНА РОБОТА 1.1. КЕРУВАННЯ ВЕРСІЯМИ ЗА ДОПОМОГОЮ GIT

Мета роботи: вивчити та закріпити на практиці можливості системи керування версіями, одержати практичний досвід у використанні Git та GitHub.

Обладнання:

- ПК IBM PC x86 CPU з встановленою операційною системою;
- доступ до мережі інтернет.

1.1 Теоретичні відомості

Система керування версіями (англ. *source code management*, SCM) – програмний інструмент для керування версіями одиниці інформації: вихідного коду програми, скрипту, веб-сторінки, веб-сайту, 3D моделі, текстового документу тощо.

Система керування версіями – це потужний інструмент, який дозволяє одночасно, без перешкод один одному, проводити роботу над груповими проектами.

Системи керування версіями зазвичай використовуються при розробці програмного забезпечення для відстеження, документування та контролю над поступовими змінами в електронних документах: у коді додатків, кресленнях, електронних моделях та інших документах, над змінами яких одночасно працюють декілька людей.

Кожна версія позначається унікальною цифрою чи літерою, зміни документу занотовуються. Зазвичай також зберігається автор зробленої зміни та її час.

Інструменти для контролю версій входять до складу багатьох інтегрованих середовищ розробки.

Системи керування версіями існують двох основних типів: з централізованим сховищем та розподіленим.

Централізовані системи контролю версій. Централізована система контролю версій (клієнт-серверна) –

система, дані в якій зберігаються в єдиному «серверному» сховищі. Весь обмін файлами відбувається з використанням центрального сервера. Є

9

можливість створення та роботи з локальними репозиторіями (робочими копіями).

Переваги:

- загальна нумерація версій;
- дані містяться на одному сервері;
- можлива реалізація функції блокування файлів;
- можливість керування доступом до файлів.

Недоліки:

– необхідність мережевого з'єднання для оновлення робочої копії чи збереження змін.

До таких систем відносять Subversion, Team Foundation Server.

Розподілені системи контролю версій. Розподілена система контролю версій (англ. Distributed Version Control System, DVCS) – система, яка використовує замість моделі клієнт-сервер, розподілену модель зберігання файлів. Така система не потребує сервера, адже всі файли знаходяться на кожному з комп'ютерів.

Переваги:

- кожний з розробників працює зі своїм власним репозиторієм;
- рішення щодо злиття віток приймається керівником проєкту;
- немає потреби в мережевому з'єднанні.

Недоліки:

- немає можливості контролю доступу до файлів;
- відсутня загальна нумерація версії файлу;
- значно більша кількість необхідного дискового простору;
- немає можливості блокування файлів.

До таких систем відносять Git, Mercurial, SVK, Monotone, Codeville, BitKeeper.

Система контролю дозволяє зберігати попередні версії файлів та завантажувати їх за потребою. Вона зберігає повну інформацію про версію кожного з файлів, а також повну структуру проєкту на всіх стадіях розробки. Місце зберігання даних файлів називають репозиторієм. В середині кожного з репозиторіїв можуть бути створені паралельні лінії розробки – гілки.

Гілки, зазвичай, використовують для зберігання експериментальних, незавершених (alpha, beta) та повністю

10

робочих версій проєкту (final). Більшість систем контролю версії дозволяють кожному з об'єктів присвоювати теги, за допомогою яких можна формувати нові гілки та репозиторії.

Використання системи контролю версії є необхідним для роботи над великими проєктами, над якими одночасно працює велика кількість розробників. Системи контролю версії надають ряд додаткових можливостей:

- можливість створення різних варіантів одного документу; – документування всіх змін (коли і ким було змінено/додано, хто який рядок змінив);

- реалізує функцію контролю доступу користувачів до файлів. Є можливість його обмеження;

- дозволяє створювати документацію проєкту з поетапним записом змін в залежності від версії;

- дозволяє давати пояснення до змін та документувати їх. **Словник основних термінів-сленгів:**

- транк (trunk) – основна гілка коду;

- бранч (branch) – відгалуження;

- чекін (Check in (submit, commit)) – відправлення коду в репозиторій;

- чекаут (Check out) – одержання зміни з репозиторію; – конфлікти – виникають, коли кілька людей правлять один і той же код, конфлікти можна вирішувати;

- патч – шматок з записаними змінами, які можна застосувати до сховища з кодом.

Керування версіями розробки програмного забезпечення проомонстровано на прикладі сервісу GitHub.

Для керування версіями розробки програмного забезпечення обрано сервіс GitHub.

Переваги:

- простий спосіб реєстрації;
- адаптовані під розповсюдженні версії операційних систем клієнти;
- простий спосіб синхронізації між сервісом та локальною версією;
- підтримка багатьох мов програмування;
- можливість відновлення репозиторію після видалення. Недоліки:

11

- для Windows необхідний NET Framework 4.0;
- ручна синхронізація;
- для проєкту що складається з декількох директорій, потрібно створити відповідно декілька репозиторіїв у системі та відслідковувати кожен з них;
- невірне відображення кирилиці після відновлення з віддаленого репозиторію.

Реєстрація у системі.

- Зайти на сайт <https://github.com/>.
- Натиснути на кнопку SING UP (див. рис. 1.1).

Рисунок 1.1 – Кнопка для реєстрації в системі

– Заповнити відповідні поля форми реєстрації (див. рис. 1.2). – Після реєстрації відкриється головна форма системи, де є можливість створити новий проєкт та завантажити клієнт відповідно до операційної системи.

– Встановлення Git. Перед початком використання Git, необхідно встановити його на локальному комп'ютері. Встановлення Git на Linux за допомогою бінарного пакету – через основний менеджер управління пакетами (для Debian-подібного дистрибутиву, такого як Ubuntu):

```
$ sudo apt-get install git
```

12

Рисунок 1.2 – Форма реєстрації учасника

– Інсталяція на Windows: є декілька шляхів встановити Git під Windows. Офіційна збірка доступна для завантаження з сайту Git. Перейдіть до <http://git-scm.com/download/win> і завантаження почнеться автоматично.

– Початкове налаштування Git. Налаштування потрібно виконати лише один раз (налаштування зберігаються між

оновленнями). Встановлення імені користувача та адреси електронної пошти (кожен коміт у Git використовує цю інформацію і вона включена у коміти):

```
$ git config --global user.name "John Doe"
```

```
$ git config --global user.email johndoe@example.com
```

Перевірка налаштувань – виконати команду `git config --list`, щоб переглянути всі налаштування, котрі Git встановив. –

Створення Git-репозиторію. Для створення Git-репозиторію використовують два основних підходи. Перший приймає існуючий проєкт або каталог і імпортує його в Git. Другий клонує існуючий репозиторій Git з іншого сервера.

Ініціалізація репозиторію в існуючому каталозі: для початку використання Git для існуючого проєкту необхідно зайти в каталог проєкту та виконати:

```
$ git init
```

13

Це створить новий підкаталог `.git`, який містить всі необхідні файли репозиторію – скелет Git-репозиторію. На даний момент, у проєкті ще нічого не відстежується.

Якщо необхідно додати існуючі файли під версійний контроль, необхідно проіндексувати ці файли і зробити перший коміт. Це можна зробити за допомогою декількох команд `git add`, що визначають файли, які необхідно відслідковувати, після чого треба виконати `git commit`:

```
$ git add *.c
```

```
$ git add LICENSE
```

```
$ git commit -m 'Перша версія проєкту'
```

Клонування існуючого репозиторію: для отримання копії існуючого Git-репозиторію (наприклад, проєкту, в якому необхідно прийняти участь) потрібна команда `git clone`. Git отримує повну копію майже всіх даних, що є у сервера. Кожна версія кожного файлу в історії проєкту витягується автоматично, коли виконується `git clone`. Якщо щось станеться з диском серверу, зазвичай можна використати майже будь-який з клонів на будь-якому клієнті щоб повернути

сервер до стану на момент клонування.

Щоб клонувати репозиторій треба використати команду `git clone [url]`. Наприклад, для клонування бібліотеки Git libgit2:
`$ git clone https://github.com/libgit2/libgit2`

Це створить директорію під назвою “libgit2”, проведе ініціалізацію директорії .git, забере всі дані для репозиторію, та приведе директорію до стану останньої версії.

Якщо необхідно зробити клон репозиторію в директорію з іншою назвою, можна задати її у наступному параметрі команди: `$ git clone https://github.com/libgit2/libgit2 mylibgit`

Ця команда робить те саме, що й попередня, тільки цільова директорія називається mylibgit.

Git має декілька різних протоколів передачі даних, які можна використовувати: `https://`, `git://` або `user@server:шлях/до/репозиторію.git`, що використовує SSH протокол.

– Створити проєкт з кількома файлами, синхронізувати його з сайтом. Якщо вже є локальний репозиторій Git і треба його викласти в загальний доступ, то спочатку необхідно створити

14

віддалений репозиторій (наприклад на GitHub), а потім виконати команди наведені нижче, змінивши відповідно назву репозиторію. Зв'язування локального репозиторію з

віддаленим [https]: `$ git remote add origin`

`https://github.com/n0tb0dy/UpRemote.git` Зв'язування локального

репозиторію з віддаленим [ssh]: `$ git remote add origin`

`git@github.com:n0tb0dy/UpRemote.git` Публікування вітки master у віддалений репозиторій:

`$ git branch -M main`

`$ git push -u origin main`

– Внести зміни до проєкту. Відобразити зміни у проєкті на сайті. Додавання всіх файлів (в тому числі і папок):

`$ git add .`

\$ git commit -m "first commit"

Встановлення зв'язку з віддаленим репозиторієм, обчислення локальних змін, відсутніх в ньому, і їх передачі у віддалений репозиторій:

\$ git push

– Внести зміни на сайті і відобразити їх локально в проєкті. Якщо гілка налаштована слідкувати за віддаленою гілкою, це можна виконати з допомогою команди `git pull` для автоматичного отримання змін та злиття віддаленої гілки до поточної гілки.

Основні команди `git` наведено в табл. 1.1, дія команд в системі `git` показана на рис. 1.3.

Таблиця 1.1 – Основні команди `git`

Команда Призначення

1 2

`git add` додає вміст робочої директорії в індекс (staging area) для подальшого `commit`

`git status` показує стану файлів в робочій директорії і індексі: які файли змінені, але не додані в індекс; які очікують комітів в індексі

`git commit` бере всі дані, додані в індекс за допомогою `git add`, і зберігає їх зліпок у внутрішній базі даних, а потім переміщує вказівник поточної гілки на цей зліпок. Ключі:

-a для додавання всіх змін в індекс без використання `git add`, -m для передачі повідомлення комітів без запуску редактора

15

Продовження табл. 1.1

1 2

`git fetch` зв'язується з віддаленим репозиторієм і забирає з нього всі зміни, яких поки немає і зберігає їх локально

`git pull` працює як комбінація команд `git fetch` і `git merge`, тобто `Git` спочатку забирає зміни із зазначеного віддаленого сховища, а потім намагається злити їх з поточної гілкою

`git push` використовується для встановлення зв'язку з віддаленим репозиторієм, обчислення локальних змін відсутніх в

ньому, і їх передачі в вищезгаданий репозиторій

Рисунок 1.3 – Команди переміщення даних Git

Детальна інформація про можливості роботи з Git – <https://git-scm.com/book/uk/v2>, основні команди – <http://www.fandroid.info/shpargalka-po-komandam-git/>.

Отже, в результаті взаємодії з системою, стало можливим відслідковувати зміни файлів проєкту. Складність системи не дозволяє повністю автоматизувати внесення змін, але слід розуміти що зміни у проєкті відбуваються під контролем розробника і він самостійно може додати до списку змін внесенні ним корективи.

16

1.2 Порядок виконання роботи

- 1 Створіть репозиторій на GitHub.
- 2 У вітку main додайте файл README.md з довільним текстом.
- 3 У відтці з назвою initialize-repository помістіть:
 - файл практичної роботи з титулкою, метою,

теоретичними відомостями;

- README.md;

- .github/PULL_REQUEST_TEMPLATE.md із секціями: Task link, What has been changed?, Important notes, Self-checks; – файл .gitignore.

4 Створіть Pull Request з initialize-repository у вітку main.

5 Додайте двох одногрупників до репозиторію як contributors, додайте цих одногрупників як reviewers, додайте себе як assignee. 6 Одержіть два підтвердження (approves).

7 Оформити звіт.

1.3 Зміст звіту

Звіт має містити:

- титульний аркуш;

- мету роботи і завдання;

- покроковий опис роботи, копії екранів з результатами виконаної роботи;

- висновки.

Запитання для самоконтролю

1 Охарактеризуйте можливості систем контролю версій. 2 Як називають основну гілку розробки проєкту?

3 За якими критеріями ділять системи контролю версій? 4 Назвіть варіанти створення репозиторію git.

5 Як додати під версійний контроль існуючі файли?

Рекомендована література

1 Офіційна документація Git [Електронний ресурс]. – Режим доступу: <https://git-scm.com/doc>

2 Chacon S. Pro Git [Електронний ресурс] / Scott Chacon, Ben Straub. – Apress, 2014. – 2nd Edition. – Режим доступу: <https://git-scm.com/book/en/v2>

3 Kedar Vijay Kulkarni. How to create a pull request in

GitHub // opensource.com [Електронний ресурс]. – Режим доступу:

<https://opensource.com/article/19/7/create-pull-request-github>

4 Creating a pull request template for your repository //

GitHub Docs [Електронний ресурс]. – Режим доступу:

<https://docs.github.com/en/communities/using-templates-to-encourage>

useful-issues-and-pull-requests/creating-a-pull-request-template-for-your-repository

Тривалість заняття: 2 год.

